Quick Sort (Partion Exchange Sort)

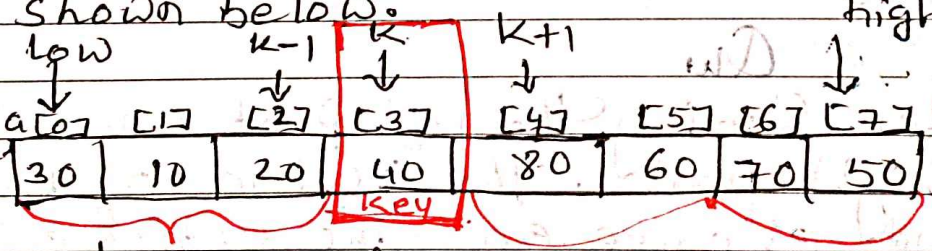
The Quick sort like Mergesort, is based on the principle of "Divide and Conquer". The Quick sort works very well on large set of data.

Design: Conside an array a consisting of 8 larger numbers 40, 80, 10, 50, 30, 20, 70, 60 the index low gives the position of the 1st element in the array and index high gives the position of last element in the array. The sorting can be done as shown below:

Step1: low > high: It indicates that there are no elements present. So, simply return the control to the calling function. The code for this case can be written as shown below:

```
if(low>high) return
```

Step2: Divide the given array of elements 40, 80, 10, 50, 30, 20, 70, 60 into two parts such that elements towards left of the key element are less than key element and elements towards right of key element are greater than key element as shown below:



elements towards left of the key less than

Elements towards right of the key are

NOTE: Assume there is a function Partition which accepts array a, low and high as parameters and divides the array into two parts as shown in figure and returns the Position of key element. So, to partition the array into two parts as shown above, call the function using the statements:

```
k ← Partition(a, low, high)
```

Step3: The left Part of the array a starts from index low to k-1, Sort the left part recursively using the Statement

```
QuickSort(a, low, k-1)
```

Step4: The right Part of the array a starts from index k+1 to high. Sort the right part recursively using the Statement.

```
QuickSort(a, k+1, high)
```

NOTE: The right part of array a starts from index k+1 to high. Sort the right part recursively using the Statement.

* After Sorting the left part and after sorting the right part, the position of key element will not change and the entire array is sorted. So, there is no need of merging as is done is done in Merge sort.

Algorithm for Quick sort

Algorithm QuickSort(a, low, high)

//Purpose: Sort the elements of the array using Quick Sort.

//Input: low: The position of the first element of array a
high: The position of the last element of array a
a: It is an array consisting of unsorted element

//output: a: It is an array consisting of sorted element

```
if(low > high) return                    // No elements to partition
k ← Partition(a, low, high)              // Divide the array into two parts
QuickSort(a, low, k-1)                   // sort the left part of the array.
QuickSort(a, k+1, high)                  // sort the right part of the array.
```

Algorithm Partition(a, low, high)

//purpose: Divide the array into parts such that elements towards left of Pivot element are less than ≤ Pivot element and element towards right of key are greater than ≥ Pivot element.

//Input: low: The position of the first element of the array
high: The position of the last element of the array
a: It is an array consisting of unsorted elements.

//output: a: Partition array such that element towards left of Pivot element are less than ≤ Pivot element and element towards right of key are greater than of key are ≥ Pivot element.

```
key ← a[low]
i ← low
j ← high+1

while (i <= j)
    do i ← i+1 while (key >= a[i])
    do j ← j-1 while (key < a[j])
        if (i < j) exchange (a[i], a[j])
end while
exchange(a[low] , a[j])
return j
```

The Main algorithm to Call the above functio
Can be written as shown below:

Algorithm that Calls QuickSort function to sort

Step1: [Input the Number of element]
read n

Step2: [Input the array elements]
for i ← 0 to n-1 do
read a[i]
end for
a[n] ← MAX-INT

NOTE: MAX-INT is

Step3: [Calls the Quick sort function]

QuickSort(a, 0, n-1)

Step4: [Output the sorted elements]

for i ← 0 to n-1 do.

Print a[i]

end for

Step5: [Terminate the algorithm]

exit

NOTE: The Time Complexity of an QuickSort Algorithm as same as Merge sort Algorithm Time Complexity – both ① Master Theorem ② Substitute Method so.

Time Complexity of an Quicksort algorithm is

$$T(n) = O(n \log_2 n).$$

$$T(n) = O(n \log n)$$

ADVANTAGES:

- In-Place: The Quick sort algorithm is in-place since it uses only a small auxiliary stack.
- Time Complexity: QuickSort Requires Time Complexity of O(n log n) in the best case and average Case to sort n items.
- It has an extremely sort inner loop.
- A very Precise Statement Can be made about-

DISADVANTAGES:

- It is not stable
- Since the algorithm is recursive, it is difficult for the beginners to grasp this algorithm requires a higher level of ledge on the part of the programmer.
- Time complexity in worst case is quadratic i.e $O(n^2)$ time.
- It is fragile, i.e, a simple mistake in implementation can go unnoticed and algorithm may not work.

Worst Case time efficiency of Quicksort

Time Worst Case occurs when, at each invocation of the procedure, the current array is partitioned into two sub arrays with one of them being empty. This situation occurs if all the elements are arranged in ascending order or descending order. for example, the array [22, 33, 44, 55] would yield the following sequence of partitions.

22 [33, 44, 55]
22 33 [44, 55]
22 33 44 [55]

To start with, 22 is the key element. It is clear from the partitioning algorithm that after partitioning, there are no elements towards left of 22 and 3 elements are there towards right of 22. In general, if an array has n elements, after partitioning, in the worst case n-1 elements will be there towards right

The recurrence relation for this case is given by

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ T(1) + T(n-1) + n & \text{Otherwise} \end{cases}$$

Where

- T(0) indicates that there are no elements towards left and it can be equal to zero.
- T(n-1) is the time required to sort remaining (n-1) elements.
- n is the time required to partition the array into two sub arrays.

Now Consider the relation,

$$T(n) = T(0) + T(n-1) + n$$
$$= T(n-1) + n \quad \text{By replacing } n \text{ by } n-1 \text{ in above eqn.}$$
$$= T(n-2) + (n-1) + n \quad \text{By replacing } n-1 \text{ by } n-2 \text{ in above eqn.}$$
$$= T(n-3) + (n-2) + (n-1) + n$$
$$\vdots$$
$$= T(n-n) + 1 + 2 + 3 \cdots (n-2) + (n-1) + n$$
$$= \frac{n(n+1)}{2.1} = \frac{n^2}{2} + \frac{n}{2} \quad \text{(By neglecting lower order terms and constants.)}$$

So Time Complexity of quicksort in the worst case is given by

$$\boxed{T(n) = O(n^2)}$$

Design: To simplify the design, assume a very large value is sorted at the end of the array. This is achieved by storing ∞ in a[n]. Apart from a, low and high the other variables that are used are:

- i : The initial value of index i is low i.e. i ← low
- j : The initial value of index j is one more than high. ie j ← high+1
- key : a[low] is treated as the key element.

The general procedure to partition the array is shown below

- Keep incrementing index i as long as key > a[i]. This is achieved using the statement do ← i+1 while(key >= a[i])
- Once the above condition fails, keep decrementing index j as long as key ≤ a[j]. This is achieved using the statement.

do j ← j-1 while(key < a[j])

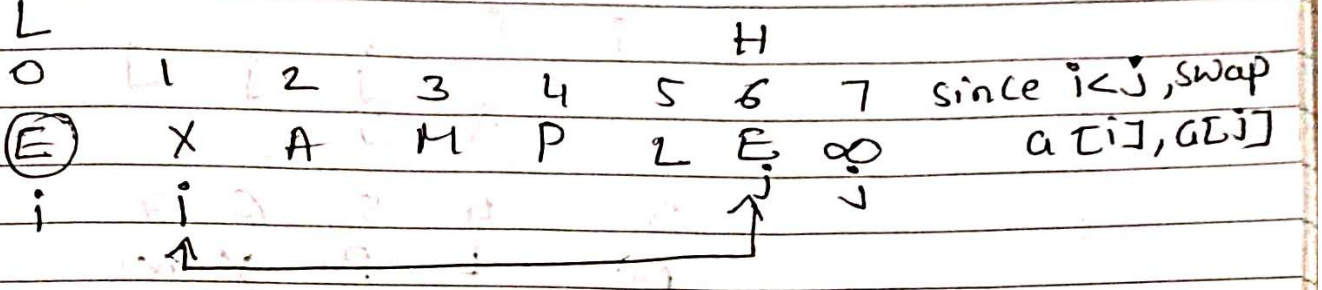
- Once the above condition fails, if i is less than j then exchange a[i] with a[j] and repeat all the above process as long as i <= j.
- Finally, exchange a[low] with a[j] and final value of j gives the position of the pivot element or key element.

EXAMPLE 1:

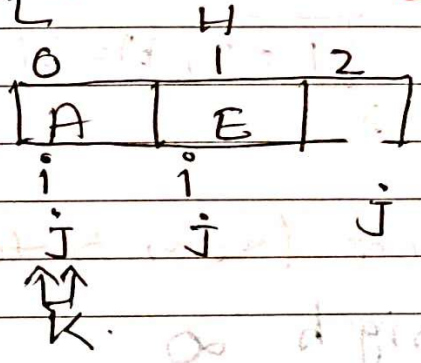
Apply Quick Sort to sort the list E, X, A, H, P, L, E in alphabetical order. Draw the tree of the recursive calls made.

Solution: The trace of Quick sort algorithm for the list E, X, A, H, P, L, E is shown below

Step1: Sort from 0 to 6



Step2: Sort from 0 to 1



After partitioning we have L=0, K=0, H=1

Step 3: Sort from 0 to -1 (it indicates that no

Step4: sort from 1 to 1 [since there is one element 1 partition is not required]

L,H
1
(E)

L=1, H=1

Step 5: Sort the element from 3 to 6

low high

3	4	5	6	7
(M)	P	L	X	∞
i	i			
		j	j	j

3	4	5	6	7
(M)	L	P	X	∞
	i	i		
	j	j		

L	(M)	P	X	∞
3	4	5	6	7

low key high

L=3, key=4
H=6

Step 6: Sort from 3 to 3 [since there is only one element no need to sort]

L=3, H=3

Step 7: Sort the element from 5 to 6

low high

P	X	∞
5	6	7

Step 8: Sort the elements from 5 to 6
[No elements exist] L = 5, H = 6

Step 9: Sort the element from 6 to 6
[Only one item exists and sorting is not required]
L = 6, H = 6

Step 10: Now, Observe that all the elements are arranged in ascending order the sorted elements are A, E, E, L, M, P, X, ∞
The tree representing the recursive calls made are shown below.

